TPG316C Unit Two Tutorial

State Management and MVVM Architecture

A COMPLETE GUIDE

Unit Two Learning Objectives

Learning Objective (LO)	DESCRIPTION
LO 1	Understand Updating the UI with State Management
LO 2	Pass State with Provider Package
LO 3	Apply Good Practices for State Management
LO 4	Understand MVVM Structure in Flutter Projects
LO 5	Separate UI and Logic Using MVVM
LO 6	Create a Simple Folder Structure for MVVM (views/, viewmodels/, models/)

KEY CONCEPTS DEFINITIONS TABLE FOR THIS TUTORIAL

Concept	**Simple Definition**	**Technical Definition**
State	The current situation of your app - what data is showing right now	Data that can change during the lifetime of the app. When state changes, the UI updates to reflect the new state.
State Management	A system that connects what the user does to what they see on screen	A pattern for handling data that changes over time and ensuring the UI reflects those changes efficiently
Model	A blueprint that defines what your data looks like	A pure Dart class that defines the structure of data with final properties and a copyWith method for immutability

ViewModel	The brain of your app - holds data and knows how to change it	A class that extends ChangeNotifier, holds a private Model instance, exposes data via getters, and contains business logic methods
View	What the user sees on screen - the UI	A StatelessWidget that displays data from the ViewModel and calls ViewModel methods in response to user actions
ChangeNotifier	A built-in Flutter class that can shout "Something changed!" to anyone listening	A mixin class from package:flutter/foundation.dart that maintains a list of listeners and notifies them when notifyListeners() is called
Provider	A package that connects your ViewModel to your View automatically	A third-party package that creates ViewModels, listens to them via ChangeNotifier, and rebuilds widgets when data changes
ChangeNotifierProvider	A widget that makes your ViewModel available to all widgets below it	A Provider widget that creates a ChangeNotifier instance and makes it available to the widget tree, handling listener registration automatically
watch()	"I need this data and want to rebuild when it changes"	A method from Provider that subscribes the current widget to a ViewModel and rebuilds the widget when notifyListeners() is called
read()	"I just need to call a method, don't rebuild me"	A method from Provider that gives direct access to a ViewModel without subscribing to changes - perfect for button presses
Consumer	"Only rebuild this specific part when data changes"	A Provider widget that rebuilds only its builder function when notifyListeners() is called, leaving the rest of the widget tree unchanged
notifyListeners()	The "shout" that tells Flutter data has changed	A method from ChangeNotifier that iterates through all registered listeners and calls them, triggering UI rebuilds
Immutability	Never change data - always create a new copy with changes	A principle where data objects cannot be modified after creation. Changes require creating a new instance with updated values
copyWith()	A helper method that creates a new copy with	A method in Model classes that returns a new instance with specified properties updated, keeping the original unchanged

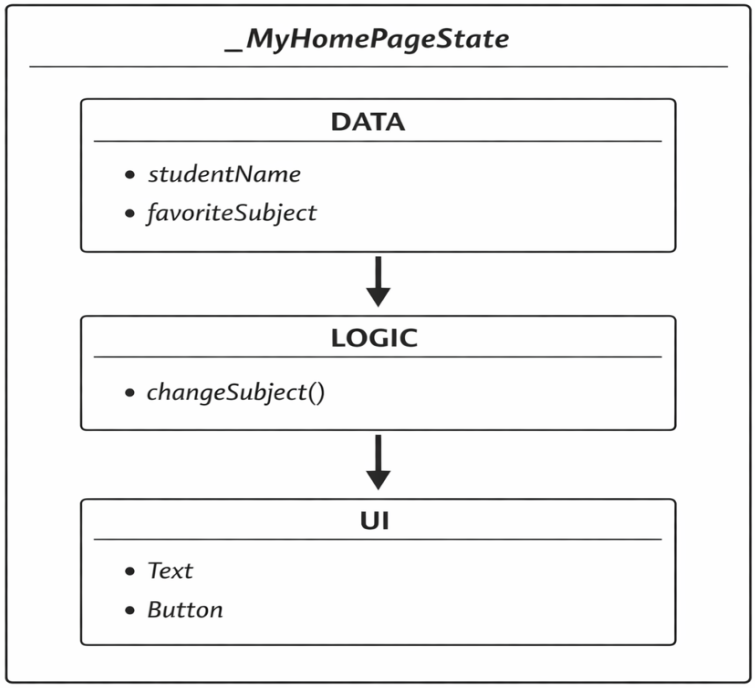
	some properties changed	
Getter	A special method that lets you read a private variable	A method that returns a private property's value, providing controlled read access without allowing direct modification
Single Source of Truth	One place where each piece of data lives	A principle where each data element is stored in exactly one location (the ViewModel's private Model) to prevent inconsistencies

PART 1: Understanding the Problem (Why We Need State Management)

The Unit 1 App: Everything in One Place

```
// Unit 1: Everything mixed together
class _MyHomePageState extends State<MyHomePage> {
  // DATA (what we show)
  String studentName = "Mpho";
  String favoriteSubject = "TPG316C";

  // LOGIC (how data changes)
  void changeSubject() {
    setState(() {
      favoriteSubject = "SOD316C";
    });
  }

  @override
  Widget build(BuildContext context) {
    // UI (what user sees)
    return Column(
      children: [
        Text(studentName),  // reads data
        ElevatedButton(
          onPressed: changeSubject,  // calls logic
        ),
      ],
    );
  }
}
```

The Problem Visualized

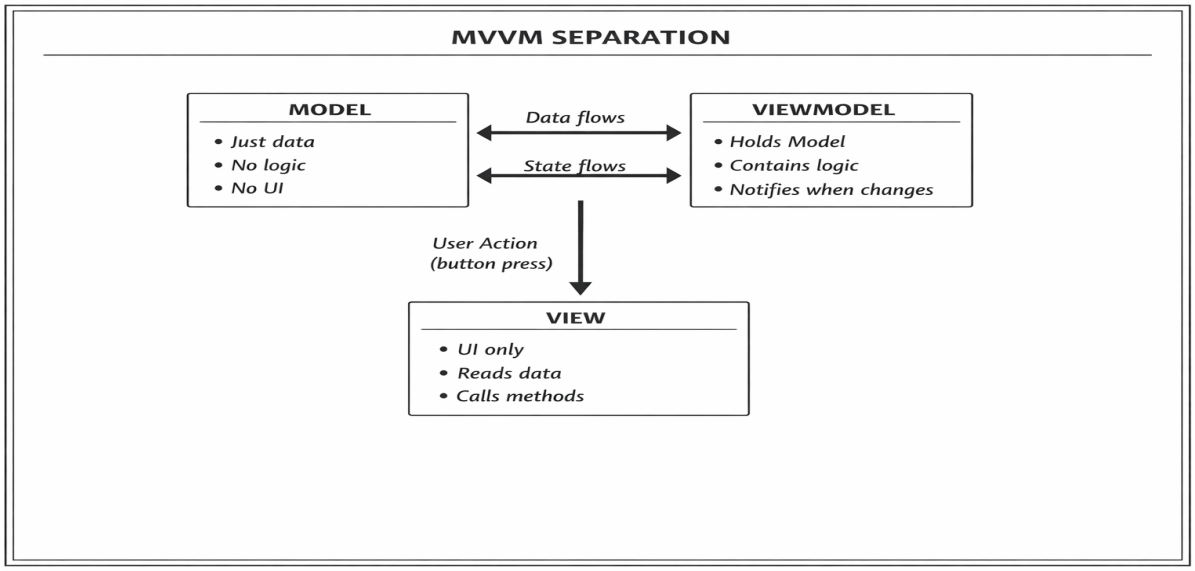


Problems with setState() - Explained

Problem	Explanation	Real-World Analogy
Tight Coupling	Data, logic, and UI are all dependent on each other. Change one, you might break others.	Like having your bedroom, kitchen, and bathroom all in one room - messy!
No Separation	Can't reuse logic in another screen without copying code	Like having to rebuild your kitchen every time you move to a new house
Hard to Test	To test if `changeSubject()` works, you must render the whole UI	Like testing if a car engine works only by driving the whole car
Poor Scalability	Adding one feature means scrolling through hundreds of lines	Like finding one book in a room where all books are piled in one corner
State Trapped	Can't access studentName from another screen	Like keeping your money only in your pocket, not in a bank

PART 2: The MVVM Solution (Separating Responsibilities)

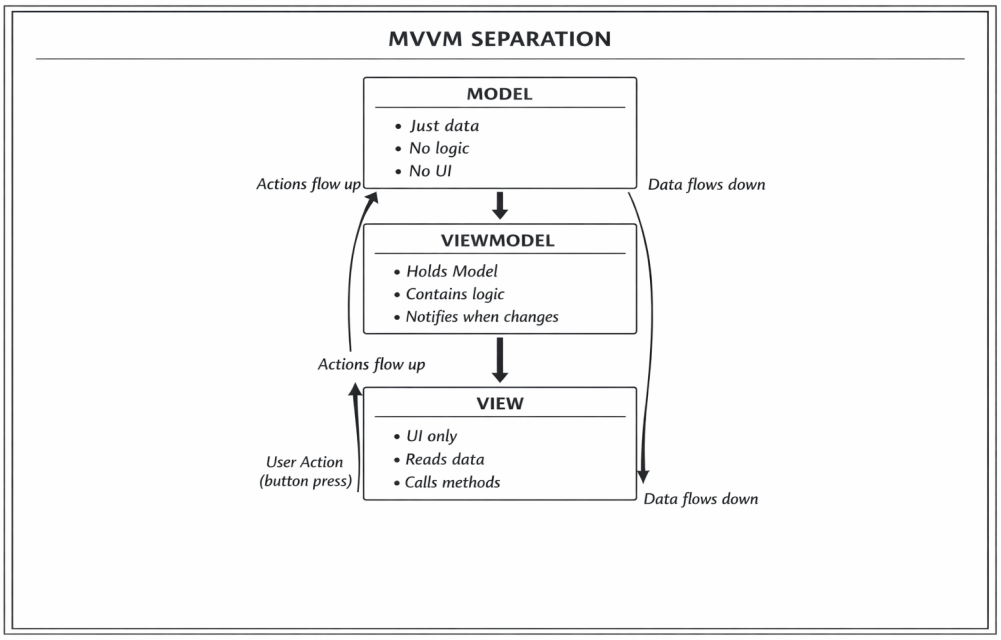
What We're Building



MVVM Components Explained

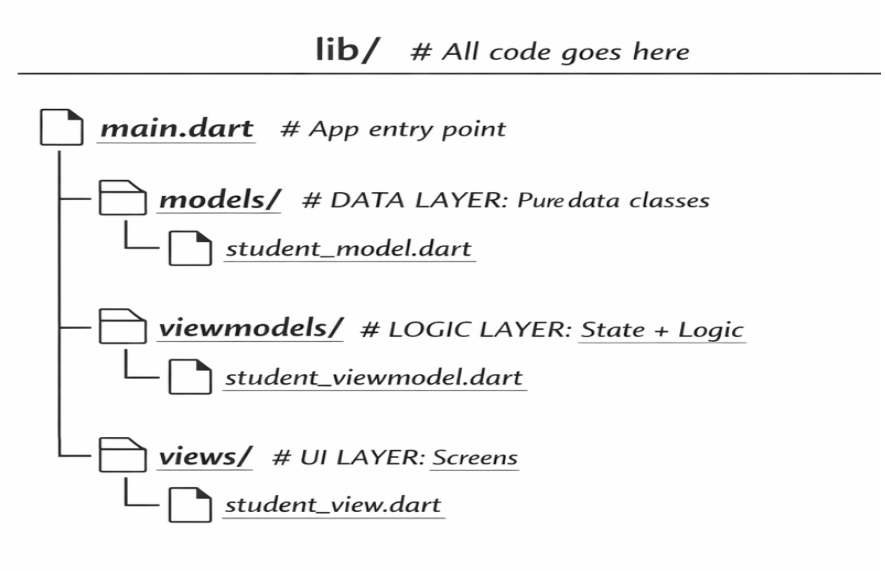
Component	Responsibility	Rules	Analogy
MODEL	Defines data structure	• Only properties & constructor • All fields `final` • Provide `copyWith()` • No Flutter imports	Like a blank form - just has spaces for name, subject, etc.
VIEWMODEL	Holds state and logic	• Extends `ChangeNotifier` • Private Model instance • Public getters • Logic methods • Call `notifyListeners()`	Like a teacher - keeps the marks, knows how to calculate, announces changes
VIEW	Displays UI	• `StatelessWidget` • Never modifies data directly • Uses `watch()`/`read()`/`Consumer` • Calls ViewModel methods	Like a chalkboard - shows current marks, lets students press buttons

The Golden Rule of MVVM



PART 3: Building MVVM Step by Step

Step 1: Create the Folder Structure



Why this structure:

Reason	Explanation
Organization	Each layer has its own folder - easy to find files
Separation	Forces you to think about where code belongs
Scalability	Add more models/viewmodels/views without chaos

Team Work	Different team members can work on different layers
Debugging	Bug in data? Check models/. Bug in logic? Check viewmodels/

COMPARISON TABLES

Unit 1 (setState) vs Unit 2 (Provider + MVVM)

Aspect	Unit 1 (setState)	Unit 2 (Provider + MVVM)
Files	1 file (main.dart)	Multiple files organized in folders
Widget Type	StatefulWidget	StatelessWidget
Where Data Lives	Inside State class	Inside ViewModel class (private)
Where Logic Lives	Inside button's onPressed	Inside ViewModel methods
How UI Updates	setState()	notifyListeners() in ViewModel
Data Access	Direct: $studentName	Via getters: ${viewModel.studentName}
Button Action	setState(() {...})	context.read<ViewModel>().method()
Testability	Difficult (need UI)	Easy (pure Dart logic)
Reusability	Low (widget-specific)	High (ViewModel can be used by multiple views)
Scalability	Breaks down as app grows	Handles complexity well

watch() vs Consumer vs read()

Feature	watch()	Consumer	read()
Syntax	`final vm = context.watch<T>()`	`Consumer<T>(builder: (ctx, vm, child) => ...)`	`context.read<T>().method()`
When to Use	When the ENTIRE widget needs data	When only PART of widget needs data	For one-time actions (button presses)
Rebuilds	Entire widget rebuilds	Only Consumer's builder rebuilds	Nothing rebuilds
Performance	Can be inefficient if widget is large	Efficient - only rebuilds what's needed	Most efficient - no rebuild
Best For	Simple widgets, top of build method	Lists, specific UI sections	Button onPressed, initState
Can Read Data?	Yes - after rebuild	Yes - inside builder	No - only for methods

PART 4: Building MVVM Step by Step

Step 2: Add Provider to `pubspec.yaml`

File: `pubspec.yaml`

```yaml
dependencies:
  flutter:
    sdk: flutter
  provider: ^6.1.5   # ADD this line
```

After adding, run in terminal:

```
flutter pub get
```

What is Provider?

Question	Answer
What is it?	A package that makes state management easier
What does it do?	It creates ViewModels, listens to them, and rebuilds UI when data changes
Why do we need it?	Without Provider, we'd have to manually manage listeners - lots of complex code
Is it the only option?	No, but it's the most popular and easiest to learn

PART 5: The MODEL (Data Layer)

File to create: `lib/models/student_model.dart`

```dart
// ============================================================
// MODEL: Defines WHAT our data looks like
//
// RULES:
// 1. Models contain ONLY properties and constructors
// 2. NO logic, NO UI, NO Flutter widgets
// 3. All properties are final (cannot change)
// 4. Provide copyWith() method for updates
// ============================================================

class StudentModel {
  // These are the pieces of data we track
  final String name;            // Student's name
  final String currentSubject;    // Currently selected subject
  final List<String> subjects;    // All available subjects
  final int currentIndex;        // Which subject is selected

  // Constructor - how we CREATE a StudentModel
  StudentModel({
    required this.name,
    required this.currentSubject,
    required this.subjects,
    required this.currentIndex,
  });

  // copyWith - how we "UPDATE" data (creates a new copy with changes)
  StudentModel copyWith({
    String? name,
    String? currentSubject,
    List<String>? subjects,
    int? currentIndex,
  }) {
    return StudentModel(
      name: name ?? this.name,
      currentSubject: currentSubject ?? this.currentSubject,
      subjects: subjects ?? this.subjects,
      currentIndex: currentIndex ?? this.currentIndex,
    );
  }
}
```

Model Rules Explained

Rule	Why It's Important	What Happens If You Break It
Properties are `final`	Data should NEVER change directly. Prevents bugs where data changes unexpectedly.	Data could be modified from anywhere - hard to track bugs
No logic in Model	Models just hold data. Logic belongs in ViewModel.	Model becomes bloated, hard to test, breaks separation
Provide `copyWith()`	The ONLY way to "change" data is to create a new copy. Old data stays unchanged (immutability).	Without it, you'd have to manually create new instances everywhere
No Flutter imports	Models are pure Dart - can be used anywhere, even outside Flutter.	Model becomes tied to Flutter, can't be used in pure Dart tests

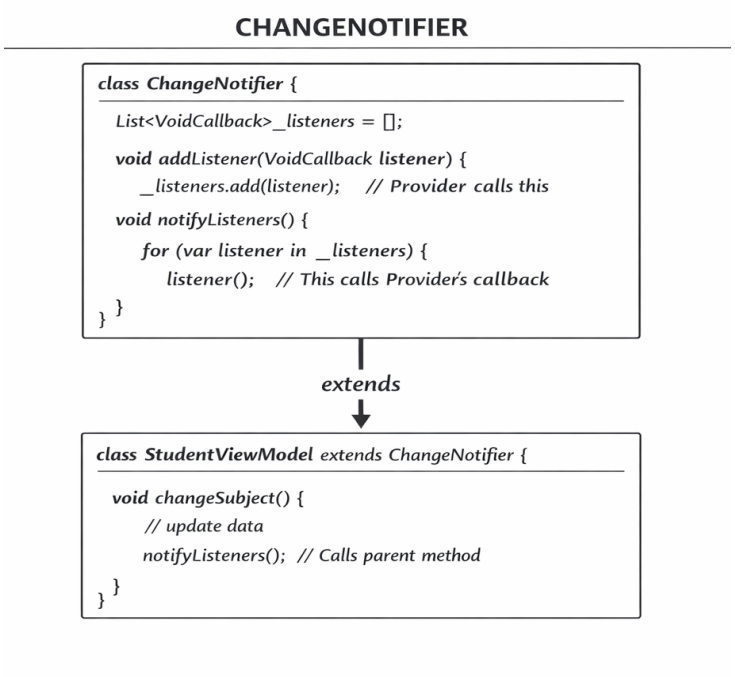
Real-world analogy: Like a passport - you don't erase and rewrite it. You get a NEW passport with updated information.

PART 6: Understanding ChangeNotifier

What is ChangeNotifier?

Question	Answer
What is it?	A class **built into Flutter** (in `package:flutter/foundation.dart`) that maintains a list of "listeners"
What does it do?	It lets objects register to be notified when something changes
Where does it come from?	Flutter SDK - NOT from Provider package!
Do we write it?	NO - we just `extends ChangeNotifier` and call `notifyListeners()`
Who listens?	**Provider package** listens automatically!

Visualizing ChangeNotifier



PART 7: The VIEWMODEL (Logic Layer)

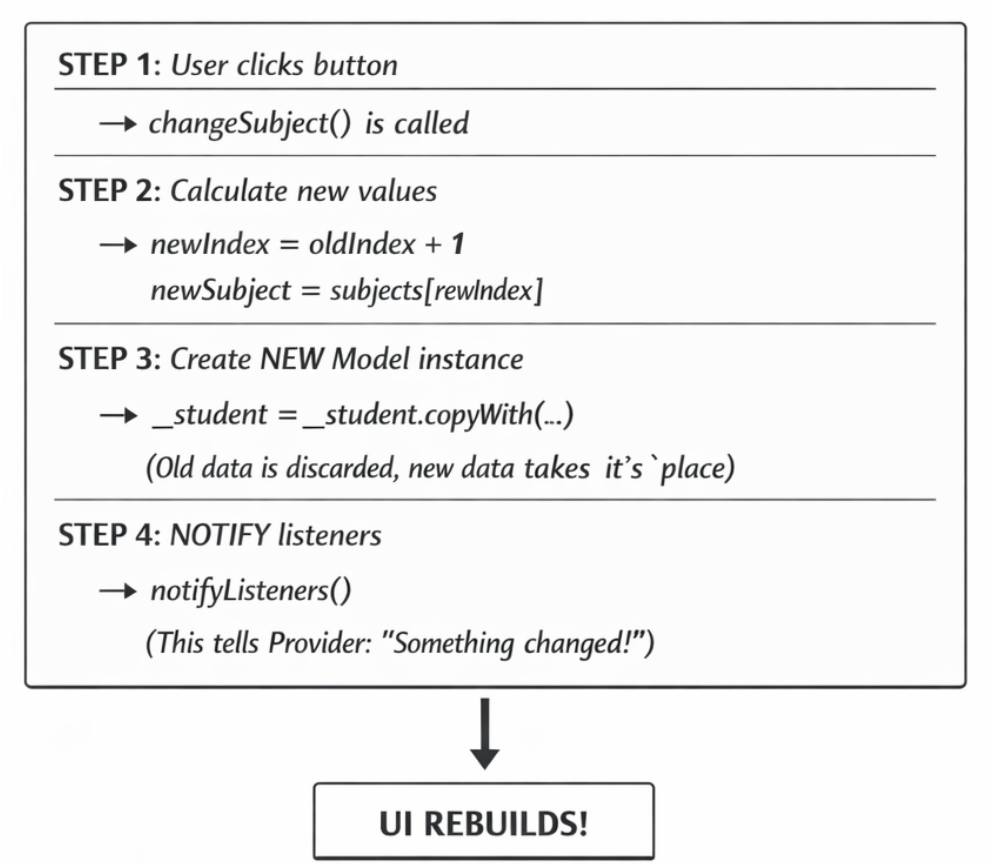
File to create: `lib/viewmodels/student_viewmodel.dart`

```dart
// ============================================================
// VIEWMODEL: Contains state and logic
//
// RULES:
// 1. ViewModel extends ChangeNotifier (from Flutter)
// 2. ViewModel holds ONE instance of Model (private)
// 3. ViewModel exposes data via getters (READ only)
// 4. ViewModel contains methods that CHANGE data
// 5. Always call notifyListeners() after changing data
// ============================================================

import 'package:flutter/foundation.dart';  // For ChangeNotifier
import '../models/student_model.dart';

class StudentViewModel extends ChangeNotifier {
  // PRIVATE Model instance - the SINGLE SOURCE OF TRUTH
  // The underscore (_) means: "This is private - don't touch from outside"
  StudentModel _student = StudentModel(
    name: "Mpho",
    currentSubject: "TPG316C",
    subjects: ["TPG316C", "SOD316C", "CMN316C", "ITS316C"],
    currentIndex: 0,
  );

  // PUBLIC GETTERS - controlled access to data
  // The View can READ but not directly WRITE
  String get studentName => _student.name;
  String get currentSubject => _student.currentSubject;
  List<String> get subjects => _student.subjects;
  int get currentIndex => _student.currentIndex;

  // LOGIC METHOD - contains the business logic
  // This is the SAME logic from Unit 1, but now in the right place
  void changeSubject() {
    // 1. Calculate new values (same as Unit 1)
    int newIndex = (_student.currentIndex + 1) % _student.subjects.length;
    String newSubject = _student.subjects[newIndex];

    // 2. Create NEW StudentModel with updated values
    _student = _student.copyWith(
      currentIndex: newIndex,
      currentSubject: newSubject,
```

ViewModel Rules Explained

Rule	Why It's Important	What Happens If You Break It
Extends ChangeNotifier	Gives the ability to notify listeners when data changes	View can't know when to rebuild
Private Model (`_student`)	Prevents View from directly modifying data	View could change data without going through logic - bugs!
Public getters	View can READ data but not WRITE	Without getters, View can't access data
Methods contain logic	All business logic in one place	Logic scattered everywhere - hard to maintain
Call `notifyListeners()`	Tells UI to rebuild with new data	UI stays on old data - user sees stale information

What Happens in the ViewModel



PART 8: The VIEW (UI Layer)

File to create: `lib/views/student_view.dart`

```dart
// ============================================================
// VIEW: Displays UI and captures user input
//
// RULES:
// 1. View is StatelessWidget (state is in ViewModel)
// 2. View NEVER contains logic - only calls ViewModel methods
// 3. View reads data via getters, never modifies directly
// 4. Use watch() for data that needs to rebuild
// 5. Use read() for actions (button presses)
// 6. Use Consumer for specific parts that need rebuilding
// ============================================================

import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import '../viewmodels/student_viewmodel.dart';
```

```dart
class StudentView extends StatelessWidget {
  const StudentView({super.key});

  @override
  Widget build(BuildContext context) {
    // METHOD 1: watch() - "I need this data and want to rebuild when it changes"
    // Use watch() when this widget DISPLAYS data that can change
    final viewModel = context.watch<StudentViewModel>();

    return Scaffold(
      appBar: AppBar(
        title: const Text("My Student Card - MVVM"),
        backgroundColor: Colors.blue,
      ),
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            // Student Info Card
            _buildInfoCard(viewModel),

            const SizedBox(height: 30),

            // METHOD 2: Consumer - rebuilds ONLY this specific part
            _buildSubjectIndicator(),

            const SizedBox(height: 20),

            // METHOD 3: read() - for button actions
            _buildChangeButton(),

            const SizedBox(height: 20),

            // Subject list using Consumer
            _buildSubjectList(),
          ],
        ),
      ),
    );
  }

  // Helper method to keep build() clean
```

```
Widget _buildInfoCard(StudentViewModel viewModel) {
  return Container(
    padding: const EdgeInsets.all(20),
    decoration: BoxDecoration(
      color: Colors.white,
      borderRadius: BorderRadius.circular(15),
      border: Border.all(color: Colors.blue, width: 2),
    ),
    child: Column(
      children: [
        Text(
          "Student Name: ${viewModel.studentName}",
          style: const TextStyle(fontSize: 24),
        ),
        const SizedBox(height: 10),
        Text(
          "Favorite Subject: ${viewModel.currentSubject}",
          style: const TextStyle(fontSize: 20, color: Colors.green),
        ),
      ],
    ),
  );
}

// Consumer example - only this part rebuilds
Widget _buildSubjectIndicator() {
  return Consumer<StudentViewModel>(
    builder: (context, viewModel, child) {
      return Container(
        padding: const EdgeInsets.symmetric(horizontal: 12, vertical: 6),
        decoration: BoxDecoration(
          color: Colors.blue[100],
          borderRadius: BorderRadius.circular(20),
        ),
        child: Text(
          "Subject ${viewModel.currentIndex + 1} of ${viewModel.subjects.length}",
          style: const TextStyle(fontSize: 16, fontWeight: FontWeight.bold),
        ),
      );
    },
  );
}
```

```
// Button using read() - doesn't rebuild itself
Widget _buildChangeButton() {
  return SizedBox(
    width: double.infinity,
    child: ElevatedButton(
      onPressed: () {
        // read() gets ViewModel without rebuilding this button
        context.read<StudentViewModel>().changeSubject();
      },
      child: const Text("Change Subject"),
    ),
  );
}

// Subject list using Consumer
Widget _buildSubjectList() {
  return Consumer<StudentViewModel>(
    builder: (context, viewModel, child) {
      return Column(
        children: [
          const Text(
            "Available Subjects:",
            style: TextStyle(fontSize: 16, fontWeight: FontWeight.bold),
          ),
          const SizedBox(height: 10),
          Wrap(
            spacing: 8,
            runSpacing: 8,
            children: viewModel.subjects.map((subject) {
              final isCurrent = subject == viewModel.currentSubject;
              return Container(
                padding: const EdgeInsets.symmetric(horizontal: 12, vertical: 6),
                decoration: BoxDecoration(
                  color: isCurrent ? Colors.green : Colors.grey[200],
                  borderRadius: BorderRadius.circular(20),
                ),
                child: Text(
                  subject,
                  style: TextStyle(
                    color: isCurrent ? Colors.white : Colors.black,
                    fontWeight: isCurrent ? FontWeight.bold : FontWeight.normal,
                  ),
```

```
),
            );
          }).toList(),
        ),
      ],
    );
  },
);
}
}
```

PART 9: Connecting Everything in main.dart

File: `lib/main.dart`

```dart
// ============================================================
// MAIN.DART: The entry point - connects everything
//
// RULES:
// 1. Wrap the app with ChangeNotifierProvider
// 2. Provider creates the ViewModel ONCE and makes it available
// 3. Use create: NOT value: for objects that should live
// ============================================================

import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import 'viewmodels/student_viewmodel.dart';
import 'views/student_view.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'Student Info App',
      theme: ThemeData(primarySwatch: Colors.blue),

      // ChangeNotifierProvider - the CONNECTOR
      // It creates the ViewModel and makes it available to all widgets below
      home: ChangeNotifierProvider(
```

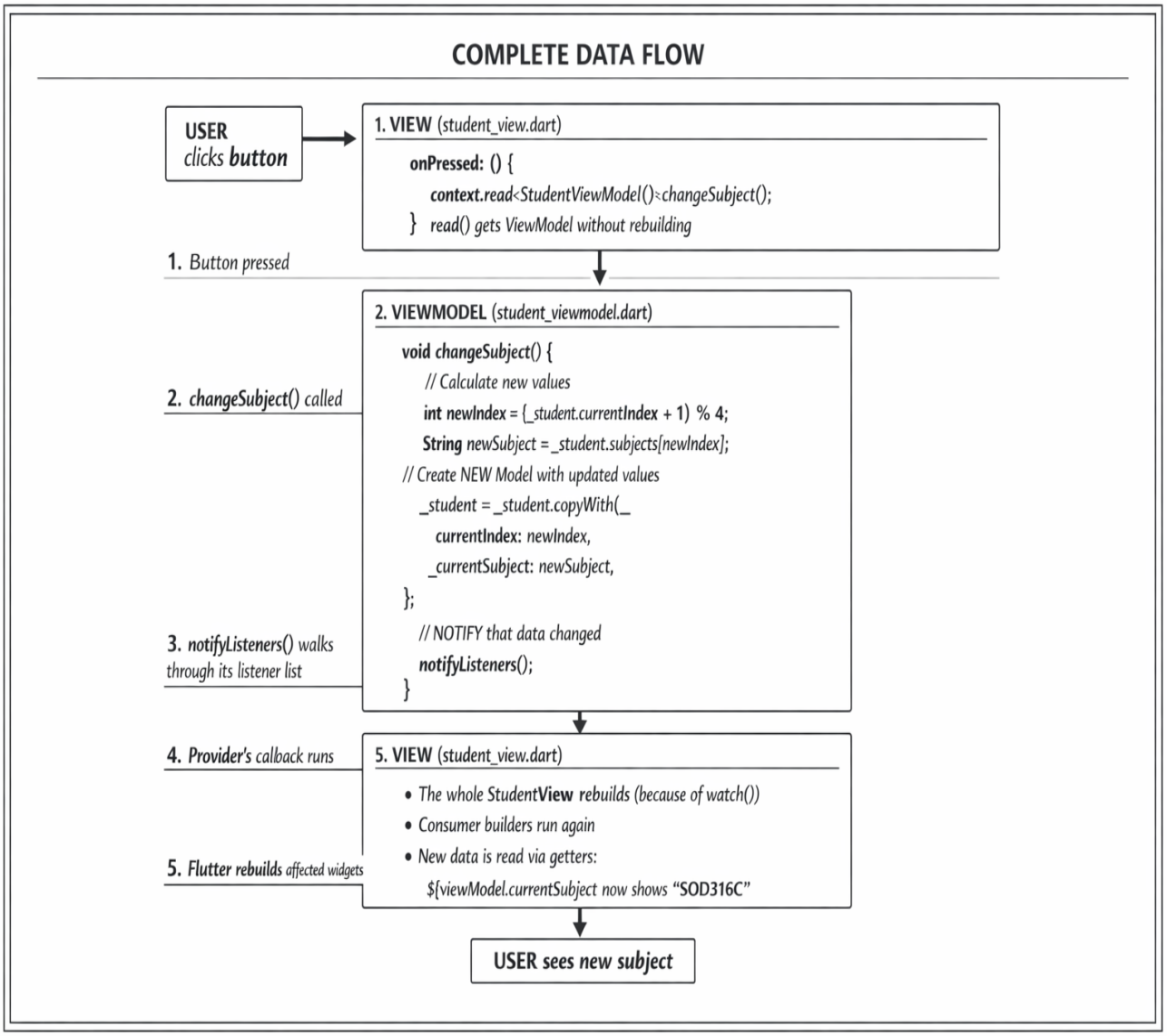
```
  // create: Creates the ViewModel ONCE when app starts
      // Important: Use create: NOT value: for objects that should live
      create: (context) => StudentViewModel(),

      // child: The first screen that can access this ViewModel
      child: const StudentView(),
    ),
  );
}
}
```

What ChangeNotifierProvider Does

Step	What Happens	Why It Matters
1. Create	Calls `create: (context) => StudentViewModel()`	Creates ONE instance of ViewModel for the whole app
2. Listen	Automatically calls `addListener()` on your ViewModel	Provider subscribes to changes
3. Provide	Makes ViewModel available to all child widgets	Any widget can access it with `watch()`/`read()`/`Consumer`
4. Rebuild	When `notifyListeners()` is called, rebuilds listening widgets	UI stays in sync with data

PART 10: The Complete Data Flow

What Happens When User Clicks "Change Subject":



PART 11: Important Rules Summary

RULES FOR MODEL (`models/`)

Rule	Why	Consequence If Broken
Only properties and the constructor	Models just hold data, nothing else	Model becomes bloated, hard to maintain
All properties `final`	Data should never change directly	Data can be modified from anywhere - bugs!
Provide `copyWith()`	The only way to "update" is to create a new copy	Have to manually create new instances everywhere

No Flutter imports	Models are pure Dart - can be used anywhere	Model tied to Flutter, can't test easily

RULES FOR VIEWMODEL (`viewmodels/`)

Rule	Why	Consequence If Broken
Extend `ChangeNotifier`	Gives ability to notify listeners	Can't tell UI when data changes
Hold ONE Model instance	Single source of truth	Multiple copies - data inconsistency
Private Model with _	Prevent direct access from View	View could modify data directly - bypass logic
Public getters	View can READ but not WRITE	View can't access data, or can modify it
Call `notifyListeners()` **after changes**	Tell UI to update	UI shows stale data
All logic in methods	Keep logic in one place	Logic scattered everywhere - hard to debug

RULES FOR VIEW (`views/`)

Rule	Why	Consequence If Broken
Use `StatelessWidget`	State is in ViewModel now	StatefulWidget with no state - confusing
Never modify data directly	Only call ViewModel methods	Data changes without logic - unpredictable
Use `watch()` **for display data**	Rebuild when data changes	UI doesn't update when data changes
Use `read()` **for actions**	Don't rebuild buttons unnecessarily	Performance wasted on unnecessary rebuilds
Use `Consumer` **for specific parts**	Optimize rebuilds	Whole screen rebuilds for small changes

RULES FOR PROVIDER USAGE

Rule	Why	Consequence If Broken
Wrap app with `ChangeNotifierProvider`	Makes ViewModel available everywhere	View can't access ViewModel

Use `create:` **not** `value:`	`create:` ensures single instance	Multiple instances - data inconsistency
One Provider per ViewModel	Each ViewModel gets its own provider	Can't access the right ViewModel
Provider at highest needed level	Put it where all consumers can access	Some widgets can't access ViewModel

PART 12: Common Mistakes and Fixes

Mistake	Wrong Code	Correct Code	Why It's Wrong
Direct data modification	`viewModel.studentName = "John"`	`viewModel.updateName("John")`	Bypasses logic, breaks encapsulation
Using `read()` **for display**	`Text(context.read<VM>().name)`	`Text(context.watch<VM>().name)`	Text won't update when data changes
Using `watch()` **in button**	`onPressed: context.watch<VM>().method`	`onPressed: context.read<VM>().method`	Button rebuilds unnecessarily
Forgetting `notifyListeners()`	`_student = newStudent;`	`_student = newStudent; notifyListeners();`	UI never updates
Putting logic in View	`if(index > 5) {...}` in button	Logic in ViewModel method	Logic can't be reused or tested
Exposing whole Model	`get student => _student`	Individual getters	View can modify Model directly
Creating ViewModel in build()	`ChangeNotifierProvider(value: VM())`	`ChangeNotifierProvider(create: (_) => VM())`	ViewModel recreated on every rebuild

Using `Consumer` every where	Everything wrapped in Consumer	Use `watch()` for simple cases	Overcomplicates simple code

PART 13: Quick Reference Card

QUICK REFERENCE CARD

📦 MODEL *(student_model.dart)*

- *class StudentModel { final ... }*
- *copyWith() for updates*
- *Pure Dart - no Flutter imports*

↓

🧠 VIEWMODEL *(student_viewmodel.dart)*

- *extends ChangeNotifier*
- *StudentModel _student (PRIVATE)*
- *getters for data (READ ONLY)*
- *methods for logic + notifyListeners()*

↓

🎨 VIEW *(student_view.dart)*

- *StatelessWidget*
- *watch() for data that rebuilds*
- ***read**() for actions*
- *Consumer for specific parts*

↓

🔌 PROVIDER SETUP *(main.dart)*

- *ChangeNotifierProvider(*

  create: (_) → StudentViewModel();

  child: StudentView(),

  }

📊 DATA FLOW

- *View calls method → ViewModel updates → **notify**Listeners()*
- *→ ChangeNotifier notifies Provider → Provider rebuilds View*
- *→ User sees new data*

🎯 CHALLENGE: Add Student Grade

The Challenge

Your app currently shows student name and favorite subject. The student also needs to track their **current grade percentage** for their favorite subject.

Requirements

Add a new feature to show the student's grade for the current subject:

1. **Model**: Add a `grade` property to `StudentModel`
2. **ViewModel**: Add methods to increase and decrease the grade
3. **View**: Display the grade and add buttons to increase/decrease it

Challenge Questions

Answer these questions to test your understanding:

1. **Where did you add the grade data and why?**
   - Answer: In Model, because it's data about the student
2. **Where did you add the logic to change grade and why?**
   - Answer: In ViewModel, because it's business logic
3. **Which method did you use to access the ViewModel in the buttons and why?**
   - Answer: `read()` because buttons don't need to rebuild
4. **What would happen if you used `watch()` instead of `read()` in the buttons?**
   - Answer: The buttons would rebuild every time grade changes - wasteful!
5. **When you click increase grade, trace the data flow:**
   - Button click → `read()` gets ViewModel → `increaseGrade()` called
   - ViewModel updates grade with `copyWith()` → `notifyListeners()`
   - Provider hears notification → View rebuilds
   - New grade displays on screen

Bonus Challenge

Add validation so grade can't go below 0 or above 100. Where would this validation go?

Summary: What You Must Remember

Concept	Memory Hook	Why It Matters
Model	"What data looks like"	Defines structure, ensures consistency
ViewModel	"Where logic lives"	Centralizes logic, makes it testable
View	"What user sees"	Pure UI, no logic, easy to modify
Provider	"The connector"	Glues everything together automatically
watch()	"Watch and rebuild"	For data that changes and needs to update
read()	"Read and run"	For one-time actions, no rebuild needed
Consumer	"Consume precisely"	For performance optimization
notifyListeners()	"Shout it out!"	Tells UI "Something changed!"

The Golden Rule:

- ***Data flows DOWN, Actions flow UP.***
- ***Never let the View** touch the Model directly.*
- ***Never put logic** in the **View.***
- *Always go through the **ViewModel.***